Actividad 4. Taller propuesto
Reglas difusas extraídas de los datos y ajuste de un sistema difuso

Informe de ejecución

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002

Fecha: 18 de octubre de 2026

Universidad Distrital Francisco José de Caldas

Índice

1. Introducción **2**

2. Marco teórico (resumen) **2**
2.1. Funciones de pertenencia . . . 2
2.2. Reglas extraídas de los datos . . . 2
2.3. Fuzzy C-Means . . . 2
2.4. Sugeno de primer orden y ANFIS . . . 3

3. Metodología **3**
3.1. Datos y particiones . . . 3
3.2. Las seis tareas . . . 3
3.3. Elección del número de etiquetas . . . 4
3.4. Ajuste del sistema difuso . . . 4

4. Resultados **4**
4.1. Características elegidas . . . 4
4.2. Función sigmoidal . . . 4
4.3. Mínimo y producto . . . 5
4.4. Matriz de confusión . . . 5
4.5. Etiquetas, reglas y exactitud . . . 6
4.6. Fuzzy C-Means y puente con MATLAB . . . 7
4.7. Ajuste con ANFIS . . . 8

5. Análisis **10**
5.1. Las cuatro medidas elegidas . . . 10
5.2. Por qué el mínimo y el producto coinciden . . . 10
5.3. De dónde salen los errores . . . 10
5.4. Etiquetas, reglas e interpretabilidad . . . 11
5.5. Agrupamiento con FCM . . . 11
5.6. Ajuste con ANFIS . . . 11

6. Conclusiones **11**

7. Declaración de uso de inteligencia artificial **12**

8. Referencias **12**

A. Código de los bloques completados **14**

B. Salida de los guiones de MATLAB **15**

1. Introducción

El cuaderno resuelto de la clase construye conjuntos difusos, un sistema Mamdani, un Sugeno de orden cero, un clasificador con reglas extraídas de Iris y un agrupamiento con Fuzzy C-Means. El taller propuesto aplica lo mismo a Wine en seis tareas, que la diapositiva del bloque 10 presenta así. Primero se eligen cuatro características solo con el entrenamiento y se programa la función sigmoidal `sigmf`. Después se calcula la fuerza de las reglas con mínimo y con producto, y se obtienen la matriz de confusión y el par más confundido. Por último se comparan etiquetas, reglas, exactitud y un árbol de decisión, y se agrupa con FCM exportando los datos para MATLAB. El enunciado del aula agrega el ejemplo de MathWorks para ajustar un sistema difuso con `tunefis`.

El aula adjunta el taller como live script de MATLAB y el cuaderno resuelto de la clase en Python. Este informe resuelve las tareas de la diapositiva en un cuaderno de Python armado sobre el resuelto, cuyas funciones se usan sin cambios, con un bloque `TODO` por tarea. Las dos piezas que dependen de MATLAB se ejecutaron en MATLAB R2026a. La primera es el ejemplo de `tunefis`, que sirve de referencia para el algoritmo ANFIS programado en Python. La segunda es el agrupamiento con `fcm` sobre los CSV que exporta el cuaderno. El cuaderno entregado se ejecutó completo con las veintiuna verificaciones en `[OK]`. La salida de los dos guiones de MATLAB está en el Anexo B.

2. Marco teórico (resumen)

2.1 Funciones de pertenencia

Un conjunto difuso asigna a cada valor un grado entre 0 y 1 (Zadeh, 1965). El cuaderno resuelto define las funciones triangular, trapezoidal y gaussiana con el orden de parámetros de MATLAB. La sigmoidal es

$$\mathrm{sigmf}(x;\,[a\ c]) = \frac{1}{1 + e^{-a(x-c)}},$$

donde c es el punto en que el grado vale 0.5 y a fija la pendiente y el sentido. Con $a > 0$ describe una etiqueta abierta hacia la derecha, como «alto» sin límite superior, y con $a < 0$ una abierta hacia la izquierda.

2.2 Reglas extraídas de los datos

El clasificador del cuaderno sigue una versión simplificada del método de Wang y Mendel (1992). Cada característica se normaliza a $[0, 1]$ con el mínimo y el máximo del entrenamiento y se cubre con n triángulos uniformes. Cada muestra de entrenamiento propone una regla con la etiqueta de mayor grado en cada característica y su clase como consecuente. Si dos muestras proponen el mismo antecedente se conserva la de mayor grado. La fuerza de una regla ante un dato nuevo combina los grados de sus antecedentes con una t-norma, la operación que representa la Y, como el mínimo o el producto. Toda t-norma es no decreciente en cada argumento, y el mínimo es la mayor de todas. La clase la decide la regla más fuerte. El número de reglas no pasa del menor entre n^F, el tamaño de la rejilla con F características, y el número de muestras.

2.3 Fuzzy C-Means

FCM (Bezdek et al., 1984) asigna a cada dato un grado de pertenencia a cada grupo, con grados que suman uno, y minimiza $J = \sum_i \sum_k u_{ik}^m \|x_k - v_i\|^2$ alternando la actualización de los centros v_i y de los grados u_{ik}, con $m = 2$. El coeficiente de partición es el promedio, sobre los datos, de la

suma de u_{ik}^2 en los grupos. Vale $1/c$ si la partición es totalmente difusa y 1 si es nítida. El índice de Rand ajustado (ARI) compara los grupos con las clases reales (Hubert y Arabie, 1985). Vale 1 si coinciden, cerca de 0 si el acuerdo es el del azar, y puede ser negativo.

2.4 Sugeno de primer orden y ANFIS

En un Sugeno de primer orden con una entrada la regla i dice «si x es A_i entonces $y_i = p_i x + r_i$», y la salida es el promedio ponderado $\hat{y} = \sum_i w_i y_i / \sum_i w_i$, con w_i el grado de x en A_i (Takagi y Sugeno, 1985). El ejemplo de MathWorks usa campanas generalizadas, $\mathrm{gbellmf}(x; [a\ b\ c]) = 1/(1 + |(x-c)/a|^{2b})$. ANFIS, sistema de inferencia difusa basado en una red adaptativa (Jang, 1993), ajusta los parámetros con un aprendizaje híbrido. Con las campanas fijas, $\hat{y}$ es lineal en los consecuentes p_i, r_i, que se resuelven por mínimos cuadrados. Con los consecuentes fijos, los parámetros a, b, c dan un paso de descenso de gradiente sobre el error cuadrático, normalizado para que su longitud sea el tamaño de paso. Ese tamaño empieza en 0.01, sube un 10 % tras cuatro reducciones seguidas del error, baja un 10 % tras dos combinaciones seguidas de una subida y una bajada, y tras cada cambio espera cuatro épocas antes de volver a evaluarse. El error se informa como RMSE, la raíz del error cuadrático medio.

3. Metodología

3.1 Datos y particiones

Wine (Aeberhard et al., 1994) describe 178 vinos de tres cultivares con 13 medidas químicas. El cuaderno resuelto lo carga desde scikit-learn y lo parte de forma estratificada con semilla 42, 124 vinos de entrenamiento y 54 de prueba, de modo que un acierto en prueba vale 1.9 puntos de exactitud. La elección de características, la normalización y las reglas usan solo el entrenamiento. La Figura 1 resume el proceso.

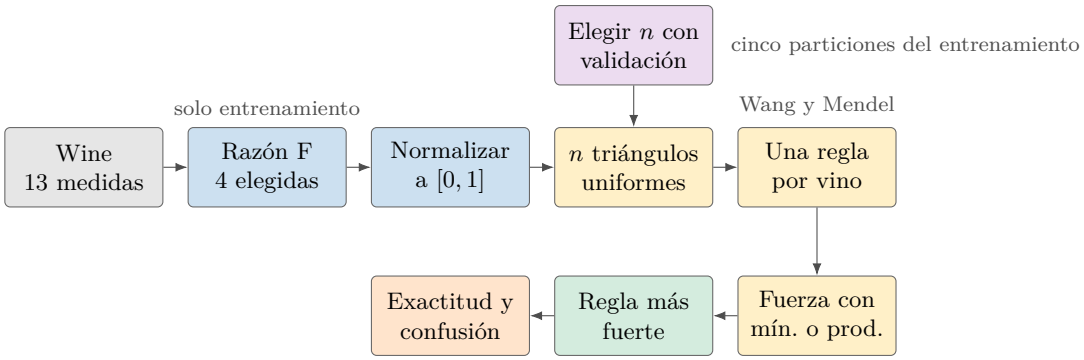


Figura 1: Flujo del clasificador difuso sobre Wine. Los bloques amarillos son la parte difusa

3.2 Las seis tareas

La primera tarea ordena las 13 medidas por la razón F del análisis de varianza calculada en el entrenamiento, que compara la dispersión de las medias de los cultivares con la dispersión dentro de cada uno, y toma las cuatro mayores. La segunda programa `sigmf` con el orden $[a\ c]$ de MATLAB y la compara con scikit-fuzzy. La tercera escribe una función de fuerza que admite las dos t-normas y compara sus predicciones con $n = 3$. La cuarta obtiene la matriz de confusión y el par (real,

predicha) con más errores. La quinta varía n entre 2 y 5, cuenta las reglas y compara la exactitud con un árbol de decisión al que se le permiten hasta tantas hojas como reglas, sobre las mismas cuatro medidas. La sexta agrupa los 178 vinos con FCM, $c = 3$ y $m = 2$, sin mirar el cultivar, compara con k-means y exporta `wine_4car_train.csv`, `wine_4car_test.csv` y el grupo de cada vino. El guion `puente_matlab_wine.m` lee esos archivos y repite el agrupamiento con `fcm`.

3.3 Elección del número de etiquetas

Elegir n mirando la prueba la convertiría en parte del ajuste. Por eso n se elige con validación cruzada estratificada de cinco particiones dentro del entrenamiento, repitiendo en cada partición la elección de características y la normalización, y el árbol se valida en las mismas particiones. Se toma el n más simple cuya exactitud media no quede más de un error estándar por debajo de la mejor. Es una adaptación del criterio de la clase, que pide el modelo más simple cuya exactitud no se distinga de la del más complejo, hecha sin usar la prueba. La validación se repite con las semillas 0 a 49 para ver si la elección depende de cómo se parte el entrenamiento. El modelo elegido se mide una vez en prueba, con el intervalo de Wilson para su exactitud y la prueba exacta de McNemar (1947) para compararlo con otros modelos sobre los mismos vinos.

3.4 Ajuste del sistema difuso

El ejemplo de MathWorks que indica el taller se abre en MATLAB con la instrucción

```
openExample('fuzzy/TuneAFuzzyInferenceSystemExample')
```

y aproxima $y = \sin(2x)/e^{x/5}$ con 101 puntos de 0 a 10. `genfis` con partición en rejilla crea un Sugeno de primer orden con cinco campanas de parámetros $a = 1.25$, $b = 2$ y centros en $0, 2.5, \ldots, 10$, y cinco reglas con consecuentes en cero. `tunefis` con el método ANFIS ajusta durante 10 épocas los 15 parámetros de las campanas y los 10 de los consecuentes. El cuaderno programa ese algoritmo en Python, con la regla del tamaño de paso tal como la describe la documentación, y compara cada época con la salida del guion `referencia_tunefis_matlab.m`. Después repite el ajuste durante 300 épocas con 100 puntos de comprobación intercalados, que no entran en el ajuste pero sí deciden qué época se conserva. El mismo guion hace esa ejecución larga con `anfis` de MATLAB.

4. Resultados

4.1 Características elegidas

La Figura 2 ordena las 13 medidas por su razón F. Las cuatro elegidas son los flavonoides (F = 187.7), la prolina (149.4), la razón OD280/OD315 de los vinos diluidos (126.6) y el alcohol (100.3). La siguiente, la intensidad de color, queda en 84.0. En el entrenamiento los flavonoides y la razón OD280/OD315 tienen una correlación de 0.77, y la prolina y el alcohol de 0.66.

4.2 Función sigmoidal

Las tres verificaciones de `sigmf` pasaron. El grado vale 0.5 en $x = c$, $\mathrm{sigmf}(5; [2\ 4]) = 1/(1 + e^{-2}) = 0.8808$ y la función coincide con `skfuzzy.sigmf`, que recibe primero el centro y después la pendiente. La Figura 3 muestra la partición uniforme que usa el clasificador y tres sigmoidales con centro en 5.

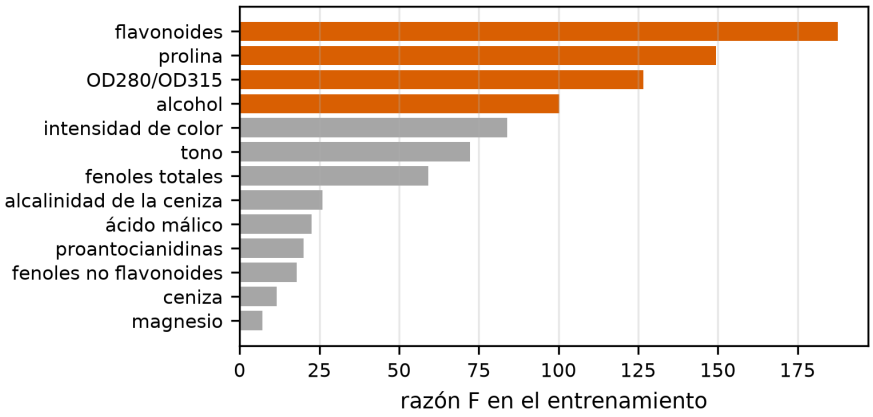


Figura 2: Razón F de cada medida en el entrenamiento. En naranja las cuatro elegidas

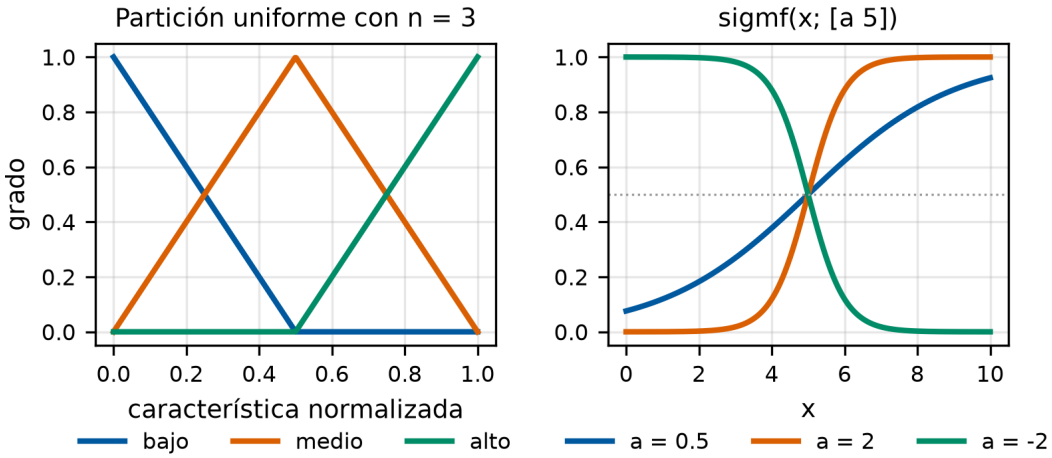


Figura 3: Izquierda, los tres triángulos de la partición uniforme. Derecha, `sigmf` con tres pendientes

4.3 Mínimo y producto

Con $n = 3$ el entrenamiento produjo 31 reglas a partir de 124 vinos. La exactitud en prueba fue 92.6 % con el producto y 92.6 % con el mínimo, y ninguna de las 54 predicciones cambió al pasar de una t-norma a la otra. Como el mínimo es la mayor t-norma, la fuerza con el producto no puede superarla, y el punto de control lo confirma en el código. En 52 de los 54 vinos de prueba existe la regla formada por la etiqueta de mayor grado en cada medida, su celda propia, y en todos ellos esa regla es la ganadora. En los otros 2 gana una regla vecina. La regla de mayor grado se lee «si flavonoides es medio y prolina es medio y OD280/OD315 es medio y alcohol es medio entonces cultivar 2».

4.4 Matriz de confusión

La Figura 4 muestra la matriz con $n = 3$, la que pide la cuarta tarea. Hubo 4 errores y el par más confundido fue cultivar 2 tomado por cultivar 3 (2 veces). A los cuatro errores los decide su celda propia. En tres de ellos los vinos de entrenamiento de esa celda son de otro cultivar, 4 de 4, 15 de 16 y el único vino que cayó en ella. El cuarto cae en la celda de la regla de mayor grado de la sección anterior, que reúne 9 vinos del cultivar 1 y 7 del 2 y tomó la clase del vino de mayor grado, del cultivar 2. Es la única de las 31 reglas cuyo consecuente no coincide con la mayoría de su celda. La

fuerza de la regla ganadora en los errores fue de 0.17 a 0.45, y en 2 de ellos supera la mediana de los aciertos, 0.26.

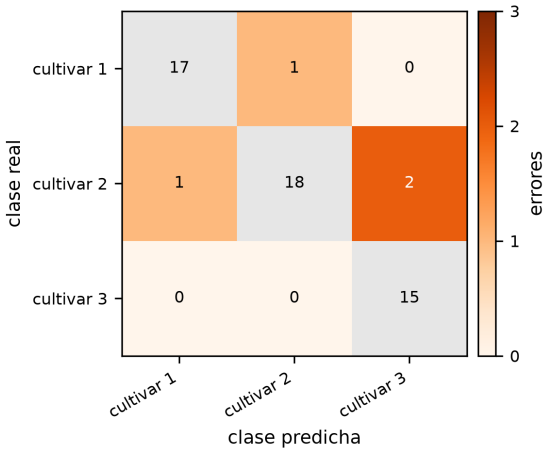


Figura 4: Matriz de confusión en prueba con $n = 3$. El color codifica solo los errores, con la misma escala que la Figura 6

4.5 Etiquetas, reglas y exactitud

La Tabla 1 y la Figura 5 reúnen la quinta tarea y la validación cruzada. El número de reglas crece con n pero se queda lejos de la rejilla completa, porque solo aparecen las combinaciones que algún vino propone. El árbol no usa más de 13 hojas porque con ellas ya separa todo el entrenamiento, y logra 87.0 % en prueba con los cuatro límites probados. En validación cruzada obtiene 91.9 ± 4.0 %.

Tabla 1: Número de etiquetas, reglas y exactitud en porcentaje. La validación cruzada es la media y la desviación muestral de cinco particiones del entrenamiento

n	Rejilla n^4	Reglas	Prueba	Validación cruzada	Árbol, prueba	Hojas
2	16	10	75.9	83.1 ± 6.5	87.0	10
3	81	31	92.6	83.9 ± 2.7	87.0	13
4	256	43	88.9	94.4 ± 4.5	87.0	13
5	625	71	92.6	89.6 ± 9.2	87.0	13

La validación cruzada favorece $n = 4$ con 94.4 ± 4.5 %. Su error estándar es 2.0 puntos y ningún n menor queda a menos de esa distancia. En las 50 repeticiones con las semillas 0 a 49, $n = 4$ tiene la mayor media en 45 y $n = 5$ en 5. Las medias son 93.6 % para $n = 4$, 86.1 % para $n = 3$ y 88.8 % para el árbol, y $n = 4$ supera al árbol en las 50. El modelo elegido, con 43 reglas, obtiene 88.9 % en prueba, con un intervalo de Wilson del 95 % entre 77.8 y 94.8 %. Sus 6 errores son todos vinos del cultivar 2, 3 tomados por el cultivar 1 y 3 por el 3, y el par más confundido empata (Figura 6). A cuatro los decide su celda propia, a uno una regla vecina porque su celda no está en la base, y uno no activa ninguna regla y recibe el cultivar 1, la clase por defecto. Las celdas que deciden esos errores tienen entre uno y tres vinos de entrenamiento. La Tabla 2 cuenta de dónde sale cada decisión según el número de etiquetas.

En prueba, el modelo elegido y el de $n = 3$ discrepan en 4 vinos. Uno lo acierta solo el primero y 3 solo el segundo, y la prueba de McNemar da $p = 0.625$. Frente al árbol discrepan en 7 vinos, 4 acertados solo por el modelo difuso y 3 solo por el árbol, con $p = 1$.

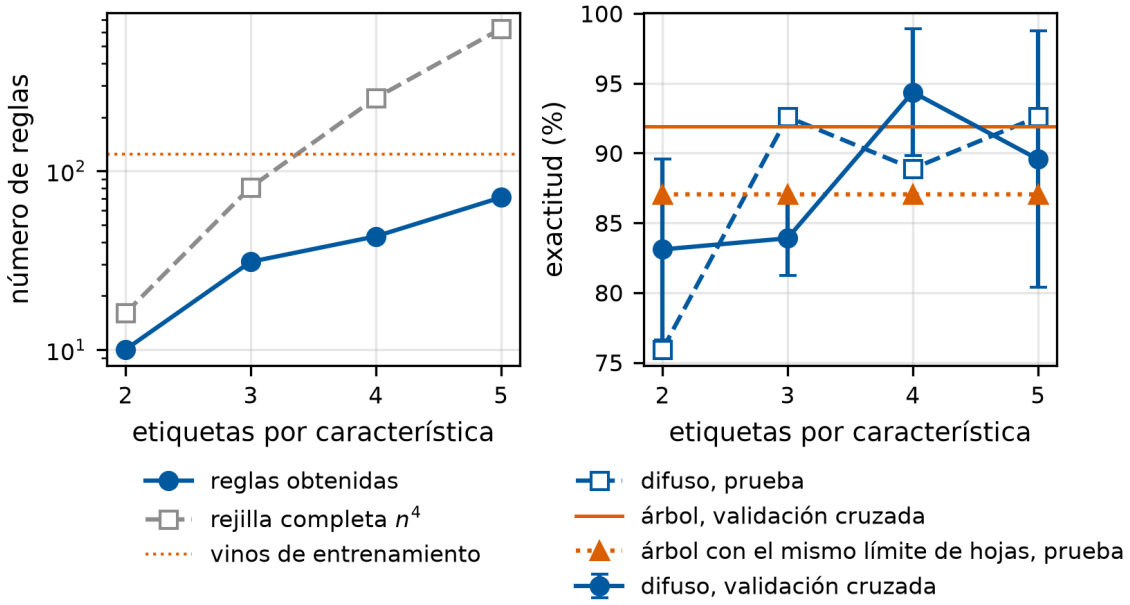


Figura 5: Izquierda, reglas obtenidas frente a la rejilla completa, en escala logarítmica. Derecha, exactitud del clasificador difuso y del árbol. Las barras son la desviación muestral de las cinco particiones

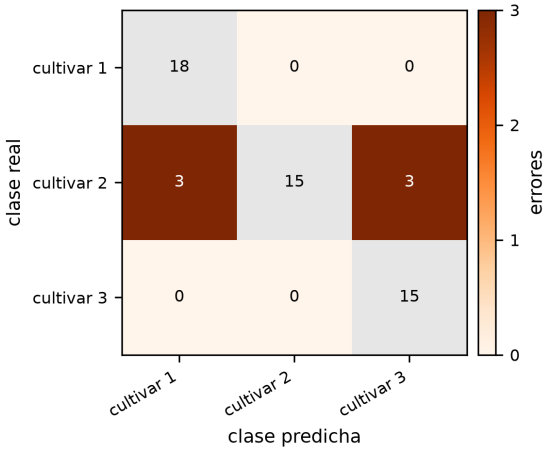


Figura 6: Matriz de confusión en prueba con $n = 4$, el número de etiquetas elegido por validación cruzada

Tabla 2: Origen de las 54 decisiones en prueba. La celda propia es la regla formada por la etiqueta de mayor grado en cada medida

n	**Celda propia**	**Otra regla**	**Ninguna regla**
2	54	0	0
3	52	2	0
4	47	6	1
5	45	7	2

4.6 Fuzzy C-Means y puente con MATLAB

FCM convergió en 19 iteraciones con $J = 7.439$. La Tabla 3 compara sus grupos con los cultivares y la Figura 7 los muestra sobre las dos medidas de mayor F. Al asignar cada vino al grupo de mayor

grado, 13 de los 178 quedan en un grupo dominado por otro cultivar, 10 de ellos del cultivar 2 en el grupo del cultivar 3. De los 20 vinos cuyo grado máximo no llega a 0.6, 14 son del cultivar 2, aunque ese cultivar es solo el 40 % de los vinos. El guion de MATLAB aplicó `fcm` con el mismo número de grupos, el mismo exponente y la misma escala. Su criterio de parada y su inicialización son los de MATLAB, pero obtuvo la misma partición, con un ARI de 1.000 frente a los grupos de Python y el mismo coeficiente de partición. Los centros coinciden a cuatro decimales salvo una coordenada, 0.7176 en Python y 0.7177 en MATLAB.

Tabla 3: Agrupamiento de los 178 vinos con las cuatro medidas elegidas, sin usar el cultivar

Método	ARI frente a los cultivares	Coeficiente de partición
FCM en Python, $m = 2$	0.794	0.690
`fcm` de MATLAB sobre los CSV exportados	0.794	0.690
k-means	0.792	–

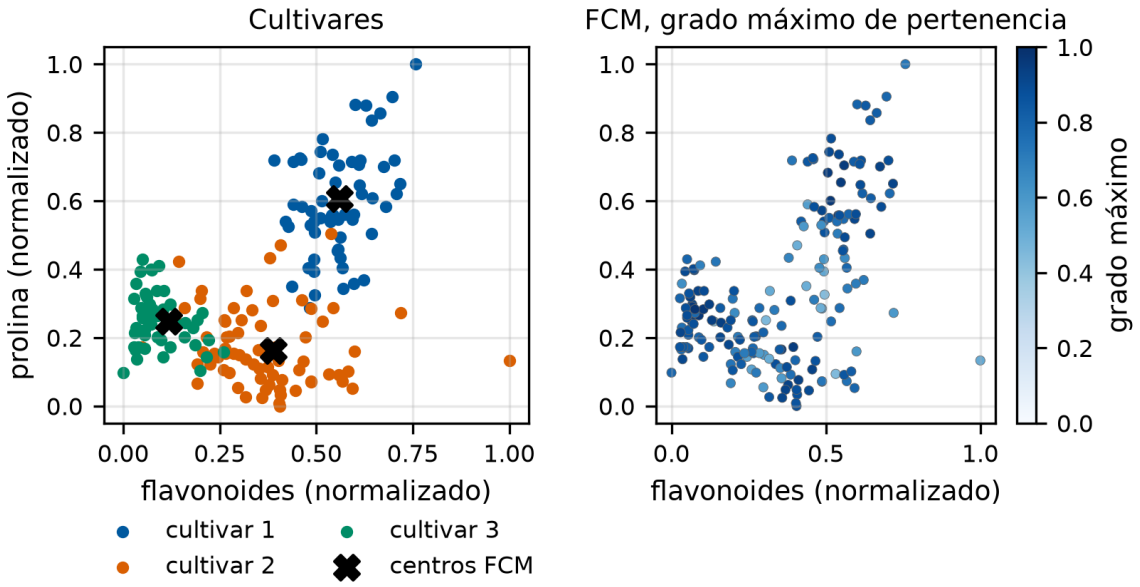


Figura 7: Izquierda, los cultivares y los centros de FCM. Derecha, el grado máximo de pertenencia de cada vino, más claro cuanto más compartido entre grupos

4.7 Ajuste con ANFIS

Las campanas iniciales de Python coinciden con las que imprime MATLAB para el sistema de `genfis`. Con los consecuentes en cero la salida es nula, así que el RMSE inicial, 0.3466485, es la raíz cuadrática media de la función que se aproxima y no dice nada de las campanas. La Tabla 4 compara las diez épocas. Las diferencias con MATLAB llegan hasta 6.0×10^{-8}, los parámetros finales de las campanas difieren en menos de 2.5×10^{-6} y los de los consecuentes hasta en 9.0×10^{-5}. El tamaño de paso subió a 0.011 tras la quinta época y a 0.0121 tras la novena, como informa MATLAB. La Tabla 5 muestra cómo se movieron las campanas y la Figura 8 la aproximación.

En la ejecución de 300 épocas los primeros 24 cambios del tamaño de paso coinciden con los de MATLAB, hasta la época 101, y después las dos secuencias se separan. Python cambia el paso 69 veces y MATLAB 61, y el paso final es 0.0013 frente a 0.0020. El RMSE de entrenamiento en la

Tabla 4: RMSE de entrenamiento por época con ANFIS en Python y con `tunefis` en MATLAB R2026a, y tamaño de paso usado en cada época

Época	Python	MATLAB	Diferencia absoluta	Paso
1	0.0694086	0.0694086	2.0×10^{-8}	0.0100
2	0.0680259	0.0680259	7.0×10^{-9}	0.0100
3	0.0666630	0.0666630	1.3×10^{-8}	0.0100
4	0.0653198	0.0653198	5.3×10^{-9}	0.0100
5	0.0639961	0.0639961	1.7×10^{-8}	0.0100
6	0.0626917	0.0626917	8.5×10^{-10}	0.0110
7	0.0612787	0.0612787	2.9×10^{-8}	0.0110
8	0.0598881	0.0598881	4.2×10^{-8}	0.0110
9	0.0585192	0.0585193	6.0×10^{-8}	0.0110
10	0.0571712	0.0571712	9.3×10^{-9}	0.0121

Tabla 5: Parámetros $[a\ b\ c]$ de las cinco campanas antes y después de las diez épocas

	Inicial (genfis)			Tras ANFIS		
Campana	a	b	c	a	b	c
1	1.2500	2.0000	0.0000	1.3136	1.9994	0.0562
2	1.2500	2.0000	2.5000	1.2414	1.9856	2.4981
3	1.2500	2.0000	5.0000	1.2423	1.9985	4.9886
4	1.2500	2.0000	7.5000	1.2755	1.9995	7.4779
5	1.2500	2.0000	10.0000	1.2511	1.9996	9.9994

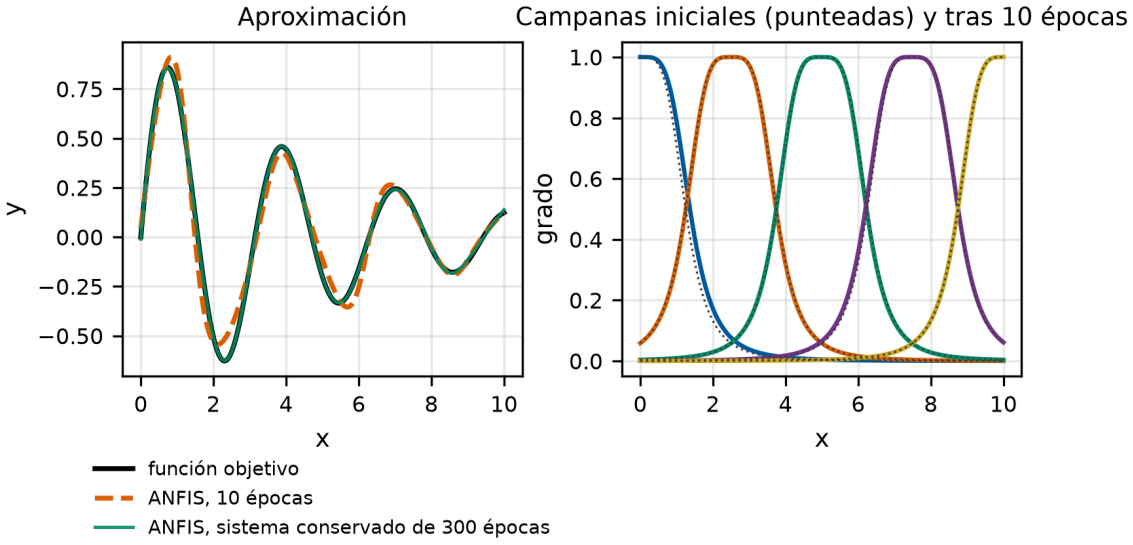


Figura 8: Izquierda, la función y las aproximaciones tras 10 épocas y del sistema conservado en 300 épocas. Derecha, las campanas iniciales y tras 10 épocas

época 300 es 0.00370 en Python y 0.00370 en MATLAB, y el de comprobación llega a su mínimo en la época 299 en los dos, 0.00333 en Python y 0.00334 en MATLAB. En el sistema conservado los parámetros a de las campanas quedan entre 1.44 y 1.64 y los centros se mueven hasta 0.28. La Figura 9 muestra las curvas de entrenamiento y de comprobación de Python.

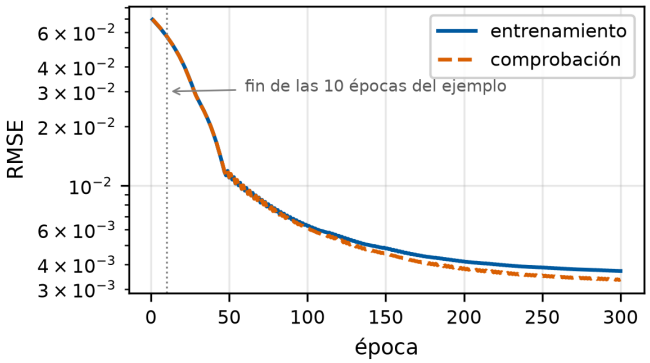


Figura 9: RMSE de entrenamiento y de comprobación durante 300 épocas de ANFIS en Python

5. Análisis

5.1 Las cuatro medidas elegidas

La razón F elige las medidas que mejor separan los cultivares una por una, pero no mira si repiten información. Los flavonoides y la razón OD280/OD315 tienen una correlación de 0.77, así que la tercera elegida repite en buena parte la información de la primera. Cuatro medidas bastan para que el clasificador difuso se quede alrededor del 90 % con decenas de reglas, frente a las 3^{13} que tendría la rejilla con las 13 medidas y tres etiquetas. La elección, la normalización y la validación cruzada se hicieron dentro del entrenamiento para que la exactitud en prueba siga midiendo lo que el modelo hace con vinos nuevos.

5.2 Por qué el mínimo y el producto coinciden

La celda propia toma en cada medida la etiqueta de mayor grado, así que sus grados son, medida por medida, mayores o iguales que los de cualquier otra regla. Como toda t-norma es no decreciente en cada argumento, si esa celda está en la base gana o empata con el mínimo, con el producto y con cualquier otra t-norma. Con $n = 3$ está en la base para 52 de los 54 vinos de prueba. En esos vinos la coincidencia es una propiedad del clasificador, y solo los otros 2 dependían de los datos, que tampoco cambiaron de clase. En general el producto se aparta del mínimo cuando hay varios grados intermedios. Con los grados (0.6, 0.6, 0.6, 0.6) el mínimo da 0.6 y el producto 0.13, mientras que con (0.5, 1, 1, 1) los dos dan 0.5. La Tabla 2 muestra que la propiedad pierde alcance cuando crecen las etiquetas, porque cada vez más vinos tienen su celda propia fuera de la base y deciden por otra regla o por la clase por defecto.

5.3 De dónde salen los errores

Todos los errores, con $n = 3$ y con el modelo elegido, involucran al cultivar 2. En la Figura 7 ese cultivar tiene flavonoides intermedios entre los del 1 y los del 3 y la prolina más baja de los tres, y comparte parte de esa zona con cada vecino. Con $n = 3$, tres de los cuatro errores caen en celdas cuyos vinos de entrenamiento son, todos o casi todos, de otro cultivar. Con estas cuatro medidas y tres etiquetas esos vinos están en la zona de su vecino, y cambiar el consecuente por la mayoría no los corregiría. Solo un error viene del consecuente, el de la celda con 9 vinos del cultivar 1 y 7 del 2, porque Wang y Mendel toma la clase del vino de mayor grado y no de la mayoría. Con más etiquetas las celdas se vacían. Con $n = 4$ las que deciden los errores tienen entre uno y tres vinos de entrenamiento, y un vino no activa ninguna regla y recibe la clase por defecto. Con $n = 5$ son dos

los vinos sin regla. Una fuerza nula delata al vino sin regla, pero en los demás errores la fuerza no avisa. Mide qué tan cerca está el vino del punto de la rejilla donde todas las etiquetas de la regla valen 1, que no tiene que ver con la fiabilidad del consecuente, y dos de los cuatro errores con $n = 3$ tienen una fuerza mayor que la mediana de los aciertos.

5.4 Etiquetas, reglas e interpretabilidad

Con dos etiquetas las reglas son pocas y se leen bien, pero el clasificador pierde exactitud. De tres a cinco etiquetas las reglas pasan de 31 a más de setenta para 124 vinos, más de una regla por cada dos vinos, y la exactitud en prueba no sube de forma sostenida. La validación cruzada y la prueba no coinciden en el mejor n. La primera prefiere 4, que tiene la mayor media en 45 de 50 repeticiones, mientras que en prueba $n = 3$ acierta dos vinos más. La prueba de McNemar no distingue esos dos modelos, ni el elegido del árbol, y el intervalo de Wilson del modelo elegido mide cerca de 17 puntos de ancho. Con 54 vinos de prueba ninguna de esas diferencias es concluyente, y el modelo que se reporta es el elegido sin mirar la prueba. En validación el difuso con $n = 4$ supera al árbol en las 50 repeticiones, 93.6 frente a 88.8 % en promedio, pero todas reparten los mismos 124 vinos y la prueba, con 54, no alcanza para confirmarlo.

5.5 Agrupamiento con FCM

FCM recupera los cultivares casi igual que k-means, con un ARI de 0.794 frente a 0.792. Además entrega el grado de cada vino en cada grupo. El cultivar 2 es el 40 % de los vinos, pero reúne 14 de los 20 vinos con grado máximo menor que 0.6, el mismo cultivar en el que se equivoca el clasificador supervisado. El coeficiente de partición de 0.690 queda entre 1/3 y 1, una partición con grupos reconocibles pero con frontera compartida. MATLAB obtiene la misma partición con otra inicialización y otro criterio de parada, así que el resultado no depende de la implementación.

5.6 Ajuste con ANFIS

En las diez épocas del ejemplo el sistema de Python sigue al de MATLAB época por época. Las diferencias en el RMSE son del orden del redondeo de los siete decimales que imprime MATLAB. Las de los consecuentes, de hasta 9.0×10^{-5}, son compatibles con que MATLAB estime los mínimos cuadrados de forma secuencial, como propone Jang (1993), mientras que Python los resuelve de forma directa. La primera época, en la que las campanas son todavía las iniciales y solo se ajustan los consecuentes, deja el RMSE en 0.0694086, y las nueve siguientes solo lo bajan a 0.0571712. Las campanas apenas se mueven, la de la izquierda se corre 0.056 y las demás menos.

En 300 épocas la secuencia de pasos de Python se separa de la de MATLAB después de la época 101, señal de que la regla programada difiere en algún detalle de la de MATLAB. Aun así el error final coincide, al menos en este ejemplo. Como la función no tiene ruido, la curva de comprobación solo muestra que el sistema interpola bien entre los puntos de entrenamiento. Además el sistema conservado se elige con el error de comprobación, y por eso su mínimo de 0.00333 no es una estimación independiente del error.

6. Conclusiones

Las seis tareas del taller propuesto quedaron resueltas en Python sobre las funciones del cuaderno resuelto, con las veintiuna verificaciones del cuaderno en `[OK]`.

Con cuatro medidas elegidas por razón F en el entrenamiento y cuatro etiquetas elegidas por validación cruzada, el clasificador con reglas extraídas de Wine obtiene 88.9 % en prueba con 43

reglas. Con tres etiquetas obtuvo 92.6 % en prueba, pero la validación estima 83.9 % para esa configuración, y McNemar no distingue los dos modelos con 54 vinos.

El mínimo y el producto dan las mismas predicciones porque la celda propia de cada vino gana con cualquier t-norma cuando existe. Todos los errores involucran al cultivar 2. Con tres etiquetas, tres de los cuatro caen en celdas cuyos vinos de entrenamiento son de otro cultivar y el cuarto en la única regla cuyo consecuente no sigue a la mayoría. Con más etiquetas los errores salen de celdas con uno a tres vinos y se suman vinos que no activan ninguna regla.

FCM recupera los cultivares con un ARI de 0.794, casi igual que k-means (0.792), y `fcm` de MATLAB sobre los CSV exportados produce la misma partición.

El algoritmo ANFIS programado en Python reproduce el ejemplo de `tunefis` de MathWorks durante las diez épocas, con diferencias de hasta 6.0×10^{-8} en el RMSE. Diez épocas mejoran sobre todo los consecuentes. Con 300 el error baja unas 15 veces y termina en el mismo valor que MATLAB, aunque la secuencia de pasos se separa después de la época 101.

7. Declaración de uso de inteligencia artificial

En este trabajo se usó Claude, asistente de inteligencia artificial de Anthropic, en tres tareas. La primera fue la redacción de partes del texto a partir de los resultados obtenidos. La segunda fue la revisión de la consistencia del documento, para que cada concepto se defina una vez y se use con el mismo sentido en todas las secciones. La tercera fue la elaboración de las gráficas a partir de las cifras que produce el código. La elaboración de los procedimientos, las modificaciones al código fuente y la maquetación estructural del documento corresponden al autor. Los resultados se verificaron con las comprobaciones automáticas del cuaderno, ejecutando los dos guiones de MATLAB sobre los mismos datos y comparando cada cifra del informe con la salida que la produce.

8. Referencias

- Aeberhard, S., Coomans, D., & de Vel, O. (1994). Comparative analysis of statistical pattern recognition methods in high dimensional settings. *Pattern Recognition*, 27(8), 1065–1077.
- Bezdek, J. C., Ehrlich, R., & Full, W. (1984). FCM: The fuzzy c-means clustering algorithm. *Computers & Geosciences*, 10(2–3), 191–203.
- Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193–218.
- Jang, J.-S. R. (1993). ANFIS: Adaptive-network-based fuzzy inference system. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(3), 665–685.
- MathWorks (2026). *Fuzzy Logic Toolbox. tunefis y el ejemplo Tune Fuzzy Inference System.* Documentación de MATLAB R2026a.
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.
- Takagi, T., & Sugeno, M. (1985). Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics*, 15(1), 116–132.
- Wang, L.-X., & Mendel, J. M. (1992). Generating fuzzy rules by learning from examples. *IEEE Transactions on Systems, Man, and Cybernetics*, 22(6), 1414–1427.

- Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338–353.

A. Código de los bloques completados

Se reproducen los seis bloques `TODO` tal como quedaron en el cuaderno entregado.

Código 1: TODO 1, cuatro características con el entrenamiento

```python
def elegir_caracteristicas(X, y, k=4):
    F, _ = f_classif(X, y)
    return np.argsort(-F)[:k], F

idx, F_tr = elegir_caracteristicas(d["Xtr"], d["ytr"])
```

Código 2: TODO 2, función sigmoidal

```python
def sigmf(x, a, c):
    """Sigmoidal [a c]: 1 / (1 + exp(-a (x - c)))."""
    x = np.asarray(x, dtype=float)
    return 1.0 / (1.0 + np.exp(-a * (x - c)))
```

Código 3: TODO 3, fuerza de las reglas con mínimo o producto

```python
def fuerza_reglas(G, R, tnorma="prod"):
    """G: N x F x n_mf grados de pertenencia. R: n_reglas x F etiquetas. Devuelve N x n_reglas."""
    grados = G[:, np.arange(G.shape[1])[None, :], R]          # N x n_reglas x F
    return grados.min(axis=2) if tnorma == "min" else grados.prod(axis=2)
```

Código 4: TODO 4, matriz de confusión y par más confundido

```python
cm = confusion_matrix(d["yte"], pred_prod)
fuera = cm.copy(); np.fill_diagonal(fuera, 0)
par = tuple(int(v) for v in np.unravel_index(np.argmax(fuera), fuera.shape))
```

Código 5: TODO 5, etiquetas, reglas, exactitud y árbol

```python
filas = []
for n in (2, 3, 4, 5):
    Rn, cn, gn = extraer_reglas(Xtr, d["ytr"], n)
    pn, _ = predecir_tnorma(Xte, Rn, cn, n)
    arbol = DecisionTreeClassifier(max_leaf_nodes=max(2, len(Rn)), random_state=SEMILLA)
    arbol.fit(d["Xtr"][:, idx], d["ytr"])
    pa = arbol.predict(d["Xte"][:, idx])
    filas.append((n, len(Rn), accuracy_score(d["yte"], pn), accuracy_score(d["yte"], pa), int(arbol
    .get_n_leaves())))
```

Código 6: TODO 6, FCM y exportación para MATLAB

```python
X_all = np.vstack([d["Xtr"], d["Xte"]])[:, idx]; y_all = np.concatenate([d["ytr"], d["yte"]])
Xa = normalizar(X_all, X_all.min(0), X_all.max(0))
centros, U, J = fcm_propio(Xa, c=3, m=2.0)
for nombre, X_, y_ in (("train", d["Xtr"], d["ytr"]), ("test", d["Xte"], d["yte"])):
    tabla = pd.DataFrame(X_[:, idx], columns=car4)
    tabla["clase"] = y_ + 1                                   # MATLAB numera desde 1
    tabla.to_csv(f"wine_4car_{nombre}.csv", index=False)
pd.DataFrame({"grupo_python": U.argmax(axis=0) + 1}).to_csv("wine_fcm_python.csv", index=False)
```

B. Salida de los guiones de MATLAB

Salida de `referencia_tunefis_matlab.m` en MATLAB R2026a, sin las líneas en blanco, la descripción de la red ni los avisos de inicio y fin del entrenamiento que imprime `tunefis`.

Código 7: Salida de referencia_tunefis_matlab.m

```
campana inicial 1  gbellmf [1.250000 2.000000 0.000000]
campana inicial 2  gbellmf [1.250000 2.000000 2.500000]
campana inicial 3  gbellmf [1.250000 2.000000 5.000000]
campana inicial 4  gbellmf [1.250000 2.000000 7.500000]
campana inicial 5  gbellmf [1.250000  2.000000 10.000000]
RMSE inicial 0.3466485
1    0.0694086
2    0.0680259
3    0.066663
4    0.0653198
Step size increases to 0.011000 after epoch 5.
5    0.0639961
6    0.0626917
7    0.0612787
8    0.0598881
Step size increases to 0.012100 after epoch 9.
9    0.0585193
10   0.0571712
Minimal training RMSE = 0.0571712
RMSE final 0.0571712
campana final 1  [1.313552 1.999443 0.056200]
campana final 2  [1.241396 1.985618 2.498113]
campana final 3  [1.242335 1.998512 4.988612]
campana final 4  [1.275471 1.999512 7.477887]
campana final 5  [1.251139 1.999575 9.999436]
consecuente final 1  [1.872801 0.215714]
consecuente final 2  [0.848484 -3.096972]
consecuente final 3  [-0.639171  2.859215]
consecuente final 4  [-0.487932  3.894183]
consecuente final 5  [0.123148 -1.032788]
300 épocas. RMSE de entrenamiento en la época 10 0.05717 y en la 300 0.00370
300 épocas. RMSE de comprobación mínimo 0.00334 en la época 299
300 épocas. Paso final 0.0020
Épocas en que cambia el paso: 6 10 14 18 22 26 30 34 38 42 46 53 57 61 65 69 73 77 81 85 89 93 97
    101 108 114 119 123 127 131 135 139 142 148 153 159 166 171 177 183 190 195 201 207 214 219 225
     232 237 243 249 256 262 267 271 275 279 283 286 292 298
```

Código 8: Salida de puente_matlab_wine.m

```
Vinos: 178 | coeficiente de partición: 0.690
ARI frente a los cultivares: 0.794
ARI frente a los grupos del FCM de Python: 1.000
Centros ordenados por la primera característica:
    0.1179    0.2474    0.1724    0.5343
    0.3883    0.1626    0.6146    0.3151
    0.5605    0.6061    0.6884    0.7177
Grupo frente a cultivar:
    57     1     0
     2    60     0
     0    10    48
```